Complete Beginner's Guide to Python & Selenium

Table of Contents

---

1. Python Basics

Variables and Data Types

Variables are like containers that store data. Think of them as labeled boxes where you put different things.

```
# String (text) - use quotes
name = "John"
message = 'Hello World'

# Integer (whole numbers)
age = 25
count = 100

# Float (decimal numbers)
price = 19.99
height = 5.8

# Boolean (True or False)
is_student = True
is_married = False
```

Easy Explanation: Variables are names you give to store information. Like putting a label on a box so you know what's inside.

Input and Output

```
# Getting input from user
name = input("What's your name? ")
age = int(input("How old are you? "))  # Convert to number

# Showing output
print("Hello", name)
print(f"You are {age} years old")  # f-string formatting
```

Operators

```
# Arithmetic operators (math)
result = 10 + 5    # Addition: 15
result = 10 - 3    # Subtraction: 7
result = 4 * 5     # Multiplication: 20
result = 10 / 3    # Division: 3.33

# Comparison operators (comparing things)
print(5 > 3)       # True
print(5 == 5)      # True (equal to)
print(5 != 3)      # True (not equal to)

# Logical operators (combining conditions)
print(True and False)  # False
print(True or False)   # True
print(not True)        # False
```

---

2. Control Structures

If, Elif, Else Statements

These help your program make decisions, like a flowchart.

```
age = 18

if age >= 18:
    print("You can vote!")
elif age >= 16:
    print("You can get a driver's license!")
else:
    print("You're still young!")
```

Easy Explanation: Like asking questions - "If this is true, do this. If not, check another condition. If nothing matches, do something else."

For Loops

Repeat something a specific number of times.

```
# Count from 1 to 5
for i in range(1, 6):
    print(f"Count: {i}")

# Go through a list
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print(f"I like {fruit}")
```

While Loops

Keep doing something while a condition is true.

```
count = 0
while count < 3:
    print(f"Count is {count}")
    count += 1  # Same as count = count + 1
```

Break and Continue

```
# Break - stop the loop completely
for i in range(10):
    if i == 5:
        break  # Stop when i is 5
    print(i)

# Continue - skip this round, go to next
for i in range(5):
    if i == 2:
        continue  # Skip when i is 2
    print(i)
```

---

3. Functions

Functions are like recipes - you give them ingredients (inputs) and they give you a dish (output).

Basic Functions

```
# Define a function
def greet(name):
    return f"Hello, {name}!"

# Call (use) the function
message = greet("Alice")
print(message)  # Output: Hello, Alice!

# Function with multiple parameters
def add_numbers(a, b):
    result = a + b
    return result

sum_result = add_numbers(5, 3)
print(sum_result)  # Output: 8
```

Lambda Functions (One-line functions)

```
# Regular function
def square(x):
    return x * x

# Lambda version (shorter)
square = lambda x: x * x

print(square(5))  # Output: 25
```

Easy Explanation: Lambda is just a quick way to write simple functions in one line.

---

4. Data Structures

Lists

Lists are like shopping lists - ordered collections of items.

```
# Creating lists
fruits = ["apple", "banana", "orange"]
numbers = [1, 2, 3, 4, 5]

# Accessing items (indexing starts from 0)
```

```
first_fruit = fruits[0]  # "apple"
last_fruit = fruits[-1]  # "orange" (negative index from end)

# Slicing (getting multiple items)
some_fruits = fruits[0:2]  # ["apple", "banana"]

# Common list methods
fruits.append("grape")      # Add to end
fruits.remove("banana")     # Remove specific item
length = len(fruits)        # Get number of items
```

Dictionaries

Dictionaries are like phone books - they connect keys to values.

```
# Creating a dictionary
person = {
    "name": "John",
    "age": 25,
    "city": "New York"
}

# Accessing values
name = person["name"]          # "John"
age = person.get("age")        # 25 (safer way)

# Adding/changing values
person["email"] = "john@email.com"
person["age"] = 26

# Getting all keys, values, or items
keys = person.keys()
values = person.values()
items = person.items()
```

Tuples and Sets

```
# Tuple - like a list but can't be changed
coordinates = (10, 20)
x, y = coordinates  # Unpacking

# Set - collection with no duplicates
unique_numbers = {1, 2, 3, 2, 1}  # Will be {1, 2, 3}
```

---

5. String Manipulation

Strings are text data. Python has many built-in methods to work with text.

```
text = "  Hello World  "

# Common string methods
cleaned = text.strip()          # Remove spaces: "Hello World"
words = text.split()           # Split into list: ["Hello", "World"]
upper = text.upper()           # Convert to uppercase
lower = text.lower()           # Convert to lowercase
replaced = text.replace("Hello", "Hi")  # Replace text

# String formatting
name = "Alice"
age = 25

# f-strings (recommended way)
message = f"My name is {name} and I'm {age} years old"

# Format method
message = "My name is {} and I'm {} years old".format(name, age)

# Old way (still works)
message = "My name is %s and I'm %d years old" % (name, age)
```

---

6. File Handling

Reading and writing files is essential for automation.

Reading Files

```
# Reading text files
with open("data.txt", "r") as file:
    content = file.read()
    print(content)

# Reading line by line
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())  # strip() removes newline characters
```

Writing Files

```
# Writing to files
```

```
with open("output.txt", "w") as file:
    file.write("Hello World\n")
    file.write("This is a new line")

# Appending to files
with open("output.txt", "a") as file:
    file.write("\nThis is appended")
```

Working with CSV Files

```
import csv

# Reading CSV
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# Writing CSV
data = [["Name", "Age"], ["Alice", "25"], ["Bob", "30"]]
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

Easy Explanation: The `with open()` statement automatically closes the file when done, like automatically closing a book when you finish reading.

---

7. Exception Handling

Exceptions are errors that happen during program execution. Handling them prevents your program from crashing.

```
# Basic try-except
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"An error occurred: {e}")
finally:
```

```
    print("This always runs, error or no error")
```

Easy Explanation: Try-except is like having a safety net. If something goes wrong in the `try` block, the program won't crash - it will run the `except` block instead.

Common Exceptions in Selenium

```
from selenium.common.exceptions import NoSuchElementException, TimeoutException

try:
    element = driver.find_element(By.ID, "button")
    element.click()
except NoSuchElementException:
    print("Element not found on the page")
except TimeoutException:
    print("Page took too long to load")
```

---

8. Modules and Packages

Modules are like toolboxes - collections of functions you can use in your programs.

Built-in Modules

```
# Time module
import time
time.sleep(2)  # Wait 2 seconds

# OS module (operating system)
import os
current_directory = os.getcwd()
print(current_directory)

# Random module
import random
random_number = random.randint(1, 10)
```

Installing External Packages

```
# In terminal/command prompt
pip install selenium
pip install pytest
pip install pandas
```

Using External Packages

```
# Import entire module
import selenium
from selenium import webdriver

# Import specific parts
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
```

---

9. Object-Oriented Programming (OOP)

OOP is like creating blueprints for objects. Think of a car blueprint - you can make many cars from one blueprint.

Classes and Objects

```
# Define a class (blueprint)
class Person:
    def __init__(self, name, age):  # Constructor
        self.name = name  # Attribute
        self.age = age    # Attribute

    def greet(self):  # Method
        return f"Hello, I'm {self.name}"

    def have_birthday(self):
        self.age += 1
        print(f"Happy birthday! Now I'm {self.age}")

# Create objects (actual instances)
person1 = Person("Alice", 25)
person2 = Person("Bob", 30)

# Use methods
print(person1.greet())  # "Hello, I'm Alice"
person1.have_birthday()  # "Happy birthday! Now I'm 26"
```

Inheritance

```
# Base class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass
```

```
# Child class inherits from Animal
class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

# Usage
my_dog = Dog("Rex")
my_cat = Cat("Whiskers")
print(my_dog.speak())  # "Rex says Woof!"
print(my_cat.speak())  # "Whiskers says Meow!"
```

Easy Explanation: Classes are like cookie cutters - you define the shape once, then make many cookies (objects) with the same shape but different flavors (attributes).

---

10. HTML and Web Elements

Before using Selenium, you need to understand web page structure.

Basic HTML Structure

```
<!DOCTYPE html>
<html>
<head>
    <title>Page Title</title>
</head>
<body>
    <h1 id="main-title">Welcome!</h1>
    <p class="description">This is a paragraph.</p>

    <div class="container">
        <button id="submit-btn" name="submit">Click Me</button>
        <input type="text" id="username" placeholder="Enter username">
    </div>

    <ul>
        <li>Item 1</li>
        <li>Item 2</li>
    </ul>
</body>
</html>
```

Using Developer Tools

1. Right-click on any element → "Inspect" or press F12
2. This shows the HTML structure
3. You can see element IDs, classes, and attributes
4. This helps you write selectors for Selenium

XPath Selectors

XPath is like giving directions to find elements on a webpage.

```
# Basic XPath examples
"//button[@id='submit-btn']"          # Button with id 'submit-btn'
"//input[@type='text']"               # Input field of type text
"//div[@class='container']//button"   # Button inside div with class 'container'
"//h1[text()='Welcome!']"             # H1 tag with exact text 'Welcome!'
"//li[contains(text(), 'Item')]"      # Li tag containing text 'Item'
```

CSS Selectors

CSS selectors are another way to find elements, often simpler than XPath.

```
# CSS selector examples
"#submit-btn"           # Element with id 'submit-btn'
".description"          # Element with class 'description'
"button[name='submit']" # Button with name attribute 'submit'
"div.container button"  # Button inside div with class 'container'
"input[type='text']"    # Input with type 'text'
```

Easy Explanation: XPath and CSS selectors are like addresses for web elements. Instead of "123 Main Street," you say "the button with id 'submit-btn'."

---

11. Selenium Fundamentals

Now let's start with Selenium automation!

Setting Up Selenium

```
# Install Selenium
pip install selenium

# Download browser driver (ChromeDriver for Chrome)
```

```
# Or use WebDriver Manager (automatic)
pip install webdriver-manager
```

Basic Selenium Setup

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
import time

# Setup Chrome driver automatically
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))

# Navigate to a website
driver.get("https://www.example.com")

# Wait a bit for page to load
time.sleep(2)

# Close browser when done
driver.quit()
```

Finding Elements

```
# Find by ID
element = driver.find_element(By.ID, "username")

# Find by Class Name
element = driver.find_element(By.CLASS_NAME, "description")

# Find by Name
element = driver.find_element(By.NAME, "submit")

# Find by XPath
element = driver.find_element(By.XPATH, "//button[@id='submit-btn']")

# Find by CSS Selector
element = driver.find_element(By.CSS_SELECTOR, "#submit-btn")

# Find multiple elements (returns a list)
elements = driver.find_elements(By.CLASS_NAME, "item")
```

Performing Actions

```
# Click on an element
```

```
button = driver.find_element(By.ID, "submit-btn")
button.click()

# Type text in input field
input_field = driver.find_element(By.ID, "username")
input_field.send_keys("my_username")

# Clear text field
input_field.clear()

# Get text from an element
heading = driver.find_element(By.TAG_NAME, "h1")
text = heading.text
print(f"Heading text: {text}")

# Get attribute value
link = driver.find_element(By.TAG_NAME, "a")
url = link.get_attribute("href")
print(f"Link URL: {url}")
```

Waits (Very Important!)

Don't use `time.sleep()` everywhere - use proper waits.

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By

# Explicit wait - wait for specific condition
wait = WebDriverWait(driver, 10)  # Wait up to 10 seconds

# Wait for element to be clickable
button = wait.until(EC.element_to_be_clickable((By.ID, "submit-btn")))
button.click()

# Wait for element to be visible
element = wait.until(EC.visibility_of_element_located((By.ID, "result")))

# Wait for element to be present
element = wait.until(EC.presence_of_element_located((By.CLASS_NAME, "content")))
```

Easy Explanation: Explicit waits are like telling Selenium "Wait until you can see the button before trying to click it" instead of "Wait 5 seconds and hope the button is there."

---

12. Advanced Selenium Topics

Handling Alerts

```
# Wait for alert and accept it
alert = WebDriverWait(driver, 10).until(EC.alert_is_present())
alert.accept()  # Click OK

# Or dismiss it
alert.dismiss()  # Click Cancel

# Get alert text
alert_text = alert.text
print(f"Alert says: {alert_text}")

# Type text in alert (if it's a prompt)
alert.send_keys("my response")
alert.accept()
```

Handling Multiple Tabs/Windows

```
# Open new tab
driver.execute_script("window.open('');")

# Get all window handles
windows = driver.window_handles

# Switch to new tab (index 1)
driver.switch_to.window(windows[1])

# Do something in new tab
driver.get("https://www.google.com")

# Switch back to original tab (index 0)
driver.switch_to.window(windows[0])
```

Handling iFrames

```
# Switch to iframe by ID
driver.switch_to.frame("iframe-id")

# Switch to iframe by element
iframe = driver.find_element(By.TAG_NAME, "iframe")
driver.switch_to.frame(iframe)

# Do something inside iframe
element = driver.find_element(By.ID, "inside-iframe")
```

```
element.click()

# Switch back to main content
driver.switch_to.default_content()
```

Taking Screenshots

```
# Take screenshot of entire page
driver.save_screenshot("screenshot.png")

# Take screenshot of specific element
element = driver.find_element(By.ID, "specific-element")
element.screenshot("element_screenshot.png")
```

Scraping Data

```
# Example: Scraping a table
table_rows = driver.find_elements(By.CSS_SELECTOR, "table tr")
data = []

for row in table_rows:
    cells = row.find_elements(By.TAG_NAME, "td")
    row_data = [cell.text for cell in cells]
    if row_data:  # Skip empty rows
        data.append(row_data)

print("Scraped data:", data)
```

Using Pytest for Testing

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By

class TestWebsite:
    def setup_method(self):
        self.driver = webdriver.Chrome()
        self.driver.get("https://www.example.com")

    def teardown_method(self):
        self.driver.quit()

    def test_title(self):
        assert "Example" in self.driver.title

    def test_login(self):
```

```
        username = self.driver.find_element(By.ID, "username")
        password = self.driver.find_element(By.ID, "password")

        username.send_keys("testuser")
        password.send_keys("testpass")

        login_button = self.driver.find_element(By.ID, "login")
        login_button.click()

        # Check if login was successful
        assert "Dashboard" in self.driver.page_source
```

Page Object Model (POM)

POM is a design pattern that makes your tests more maintainable.

```
# pages/login_page.py
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

    # Locators
    USERNAME_FIELD = (By.ID, "username")
    PASSWORD_FIELD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-btn")
    ERROR_MESSAGE = (By.CLASS_NAME, "error")

    def enter_username(self, username):
        element = self.wait.until(EC.visibility_of_element_located(self.USERNAME_FIELD))
        element.clear()
        element.send_keys(username)

    def enter_password(self, password):
        element = self.wait.until(EC.visibility_of_element_located(self.PASSWORD_FIELD))
        element.clear()
        element.send_keys(password)

    def click_login(self):
        button = self.wait.until(EC.element_to_be_clickable(self.LOGIN_BUTTON))
        button.click()

    def get_error_message(self):
```

```
        error = self.wait.until(EC.visibility_of_element_located(self.ERROR_MESSAGE))
        return error.text

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()

# test_login.py
from pages.login_page import LoginPage

def test_valid_login():
    driver = webdriver.Chrome()
    driver.get("https://www.example.com/login")

    login_page = LoginPage(driver)
    login_page.login("valid_user", "valid_pass")

    # Add assertion for successful login
    assert "Dashboard" in driver.page_source

    driver.quit()
```

Easy Explanation: POM is like organizing your tools in a toolbox. Instead of having screwdrivers scattered everywhere, you keep all screwdrivers in one section of the toolbox.

Complete Example: Web Automation Script

Here's a complete example that combines everything:

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException, NoSuchElementException
import time
import csv

class WebScraper:
    def __init__(self):
        # Setup driver with automatic driver management
        self.driver = webdriver.Chrome(
            service=Service(ChromeDriverManager().install())
```

```
    )
    self.wait = WebDriverWait(self.driver, 10)
    self.data = []

def search_products(self, search_term):
    try:
        # Go to website
        self.driver.get("https://www.example-shop.com")

        # Find search box and search
        search_box = self.wait.until(
            EC.presence_of_element_located((By.ID, "search"))
        )
        search_box.clear()
        search_box.send_keys(search_term)

        # Click search button
        search_button = self.driver.find_element(By.ID, "search-btn")
        search_button.click()

        # Wait for results to load
        self.wait.until(
            EC.presence_of_element_located((By.CLASS_NAME, "product"))
        )

        # Scrape product information
        products = self.driver.find_elements(By.CLASS_NAME, "product")

        for product in products:
            try:
                name = product.find_element(By.CLASS_NAME, "product-name").text
                price = product.find_element(By.CLASS_NAME, "price").text

                self.data.append({
                    'name': name,
                    'price': price,
                    'search_term': search_term
                })

            except NoSuchElementException:
                print("Some product information missing, skipping...")
                continue

        print(f"Found {len(products)} products for '{search_term}'")

    except TimeoutException:
        print("Page took too long to load or elements not found")
    except Exception as e:
```

```
            print(f"An error occurred: {e}")

    def save_to_csv(self, filename="products.csv"):
        with open(filename, 'w', newline='', encoding='utf-8') as file:
            if self.data:
                fieldnames = self.data[0].keys()
                writer = csv.DictWriter(file, fieldnames=fieldnames)
                writer.writeheader()
                writer.writerows(self.data)
                print(f"Data saved to {filename}")
            else:
                print("No data to save")

    def close(self):
        self.driver.quit()
        print("Browser closed")

# Usage
if __name__ == "__main__":
    scraper = WebScraper()

    try:
        # Search for different products
        search_terms = ["laptop", "smartphone", "headphones"]

        for term in search_terms:
            scraper.search_products(term)
            time.sleep(2)  # Be nice to the server

        # Save results
        scraper.save_to_csv("scraped_products.csv")

    finally:
        scraper.close()
```

Best Practices and Tips

1. Always Use Proper Waits

```
# ❌ Bad - using time.sleep everywhere
time.sleep(5)
element = driver.find_element(By.ID, "button")

# ✅ Good - using explicit waits
element = WebDriverWait(driver, 10).until(
```

```
    EC.element_to_be_clickable((By.ID, "button"))
)
```

2. Handle Exceptions Properly

```
try:
    element = driver.find_element(By.ID, "might-not-exist")
    element.click()
except NoSuchElementException:
    print("Element not found, continuing with alternative action")
    # Handle the missing element gracefully
```

3. Use Page Object Model for Larger Projects

- Keep locators in one place
- Make tests more readable and maintainable
- Easier to update when UI changes

4. Make Your Selectors Robust

```
# ❌ Fragile - depends on page structure
driver.find_element(By.XPATH, "/html/body/div[3]/div[2]/button")

# ✅ Better - uses meaningful attributes
driver.find_element(By.ID, "submit-button")
driver.find_element(By.CSS_SELECTOR, "button[data-test='submit']")
```

5. Always Clean Up Resources

```
try:
    # Your automation code here
    pass
finally:
    driver.quit()  # Always close the browser
```

Common Beginner Mistakes to Avoid

1. **Not using waits properly** - Always wait for elements to be ready
2. **Using fragile locators** - Avoid XPath based on position
3. **Not handling exceptions** - Always expect things might go wrong
4. **Forgetting to close the browser** - Use `driver.quit()`
5. **Hard-coding everything** - Use variables and configuration files
6. **Not checking if elements exist** - Elements might not always be there

---

Next Steps

1. **Practice with real websites** - Start with simple tasks
2. **Learn about different browsers** - Firefox, Edge, Safari drivers
3. **Explore advanced topics** - Headless browsing, parallel execution
4. **Learn about CI/CD** - Running tests automatically
5. **Study performance testing** - Load testing with Selenium Grid
6. **Mobile testing** - Appium for mobile app automation

Useful Resources

- **Selenium Documentation**: https://selenium-python.readthedocs.io/
- **Python Documentation**: https://docs.python.org/3/
- **Pytest Documentation**: https://pytest.org/
- **CSS Selector Reference**: https://www.w3schools.com/cssref/css_selectors.asp
- **XPath Tutorial**: https://www.w3schools.com/xml/xpath_intro.asp

---

Summary

This guide covers everything you need to start with Python and Selenium automation:

1. **Python fundamentals** - Variables, loops, functions, data structures
2. **Web basics** - HTML, CSS selectors, XPath
3. **Selenium core concepts** - Finding elements, performing actions, waits
4. **Advanced techniques** - POM, testing with pytest, exception handling
5. **Best practices** - Writing maintainable, robust automation code

Remember: Start small, practice regularly, and don't try to learn everything at once. Begin with simple scripts and gradually add complexity as you become more comfortable.

Good luck with your automation journey! 🚀